Facial Emotion Recognition using CNN on FER2013

A Custom Convolutional Neural Network for Seven-Class Facial Expression Classification

By

Isaac Ebu-Danso · Alhagie Kijera · Selvi Putri · Mahsa Eftekhari

990217587·990215869·990215892·990215809

Kühne Logistics University

Machine Learning and Deep Learning Course

2026

GitHub repository: https://github.com/Ebu-Danso/Facial-Emotion-Recognition-using-CNN-/tree/main

KÜHNE LOGISTICS UNIVERSITY

Table of Contents

Abstract

1. Introduction

2. Background and Problem Complexity

3. Dataset and Preprocessing

4. Methodology and Implementation Workflow

5. CNN Model Architecture

6. Training Configuration

7. Results and Training Dynamics

8. Confusion Matrix and Per-Class Analysis

9. Discussion

10. Limitations

11. Ethical Considerations

12. Future Work

13. Conclusion

References

Abstract

This report presents a supervised deep learning project for facial expression classification using the FER2013 dataset. The project implements a custom Convolutional Neural Network (CNN) in TensorFlow/Keras to classify 48×48 greyscale facial images into seven expression labels: angry, disgust, fear, happy, neutral, sad, and surprise. The dataset contains 35,887 images, divided into 28,709 training images and 7,178 held-out test images. Images were loaded using Keras ImageDataGenerator with pixel-value normalisation through rescale=1.0/255, which maps greyscale intensity values from the original [0, 255] range to [0, 1]. No data augmentation was used in the final implementation, which is important when interpreting the observed overfitting.

The model consists of three convolutional blocks with increasing filter depth from 64 to 128 to 256 filters. Each block combines Conv2D layers, Batch Normalisation, MaxPooling, and Dropout. The convolutional feature extractor is followed by a fully connected classification head and a seven-unit Softmax output layer. Training used categorical cross-entropy loss, the Adam optimiser, EarlyStopping with patience of seven epochs, and ModelCheckpoint to preserve the best validation model. In the final saved evaluation run, training stopped at approximately epoch 47 and achieved 64.41% test accuracy with a test loss of 1.44. Training accuracy reached around 90%, while held-out test accuracy stabilised around 64%, indicating clear overfitting.

The results suggest that the model learned useful discriminative visual patterns, particularly for visually distinctive expressions such as happy and surprise. However, the confusion matrix shows weaker performance for ambiguous or visually overlapping classes, especially fear. The report therefore interprets the system as a FER2013 facial expression label classifier rather than a general detector of internal emotional states. Limitations include the absence of augmentation, lack of class weighting, training from scratch, no fixed random seed, low-resolution images, label subjectivity, and possible demographic or cultural bias. Future work should prioritise augmentation, class balancing, multiple-seed evaluation, transfer learning, explainability, face alignment, and careful ethical validation.

1. Introduction

Facial expression classification is a computer vision task in which a model assigns an expression label to an image of a human face. In machine learning contexts, it is usually formulated as

supervised multi-class classification: given an input image, the model predicts one label from a predefined set. In this project, the model classifies images from the FER2013 dataset into seven categories: angry, disgust, fear, happy, neutral, sad, and surprise. FER2013 became widely known through the "Challenges in Representation Learning" facial expression recognition challenge, which helped establish it as a common benchmark for teaching and research in visual representation learning (Goodfellow et al., 2015).

The practical appeal of facial expression classification is clear. Systems that classify visible facial expressions may support human-computer interaction, assistive technologies, driver monitoring, educational research, or affective computing experiments. Nevertheless, the task must be framed cautiously. A facial expression label is not equivalent to a verified internal emotional state. A person's face may not fully reveal what they feel, and facial behaviour is shaped by context, culture, individual differences, display rules, and voluntary expression. Ekman and Friesen (1978) provided a structured method for describing facial movement through the Facial Action Coding System, while Barrett (2017) argues that emotions should not be reduced to universal facial signatures. For this reason, this report uses careful language: the model classifies facial expression labels from FER2013 rather than detecting true emotions.

The aim of the project was to build a complete, reproducible deep learning baseline for FER2013 using a custom CNN implemented in TensorFlow/Keras. The project intentionally focuses on a course-level architecture rather than a state-of-the-art transfer-learning system. This makes the experiment useful for understanding convolutional feature extraction, training dynamics, regularisation, and evaluation. The final saved model achieved 64.41% test accuracy and 1.44 test loss. This is a reasonable result for a custom CNN trained from scratch, but it is not a state-of-the-art result and should not be interpreted as human-level emotional understanding.

2. Background and Problem Complexity

Facial expression classification may appear straightforward because humans often interpret faces quickly and intuitively. In practice, it is a challenging machine learning problem. One difficulty is that several categories are visually similar. The difference between sadness, fear, anger, and neutrality can depend on small changes in the eyes, eyebrows, mouth corners, and facial tension. These details are difficult to capture in FER2013 because each image is only 48×48 pixels. At this resolution, fine-grained texture and muscle movement may be compressed or lost.

A second difficulty is label subjectivity. FER2013 uses discrete expression categories, but facial behaviour is not always discrete. A face may be ambiguous, blended, posed, suppressed, culturally shaped, or dependent on context that is not visible in the image. Human annotators may therefore disagree about whether a face should be labelled fear, sad, neutral, or surprise. A supervised model trained on these labels learns to reproduce the dataset's annotation patterns. It does not gain direct access to the person's internal state.

A third difficulty is dataset imbalance. As shown later in Figure 1, the happy class is much larger than the disgust class. Standard categorical cross-entropy treats each image as one training example, so majority classes contribute more frequently to gradient updates. Without class weighting or oversampling, the model may learn majority classes more reliably than minority classes. Overall accuracy can therefore hide class-specific weaknesses.

Finally, facial expression classification raises broader methodological and ethical questions. CNNs can learn useful spatial features because their filters exploit local image structure, but they can also learn dataset-specific artefacts. A model may perform reasonably on a held-out test set from the same dataset while generalising less well to images collected under different lighting, camera conditions, demographic distributions, or cultural contexts. These concerns are especially important when a system involves human faces and affective labels.

3. Dataset and Preprocessing

3.1 Dataset source and structure

The project uses the FER2013 dataset from Kaggle. The dataset contains 48×48 greyscale face images labelled into seven expression categories. The version used in the project is organised into training and test directories, with images stored in class-labelled folders. This directory structure is compatible with TensorFlow/Keras image loading utilities and supports a reproducible supervised-learning workflow because labels are inferred from folder names. Kaggle describes the dataset as consisting of 48×48 pixel greyscale images of faces that are approximately centred in the image frame (Kaggle, n.d.).

The complete dataset contains 35,887 images. The training set contains 28,709 images and the test set contains 7,178 images. The seven classes are angry, disgust, fear, happy, neutral, sad, and surprise. The held-out test set was used to assess generalisation after training. In the project

figures and code, this held-out curve is referred to as a test or validation curve depending on the script context; in this report, it is interpreted as the held-out generalisation curve associated with the project's test split.

Table 1. FER2013 class distribution used in the project

Class	Training images	Test images	Total
Angry	3,995	958	4,953
Disgust	436	111	547
Fear	4,097	1,024	5,121
Happy	7,215	1,774	8,989
Neutral	4,965	1,233	6,198
Sad	4,830	1,247	6,077
Surprise	3,171	831	4,002
Total	28,709	7,178	35,887

3.2 Class distribution and imbalance

Table 1 and Figure 1 show that the dataset is not evenly distributed across classes. The happy class is the largest, with 7,215 training images and 1,774 test images, while disgust is the smallest, with only 436 training images and 111 test images. This imbalance is important because a model trained without class weighting receives many more learning signals from common classes than from rare classes. As a result, performance on the minority class may be unstable and difficult to interpret robustly.

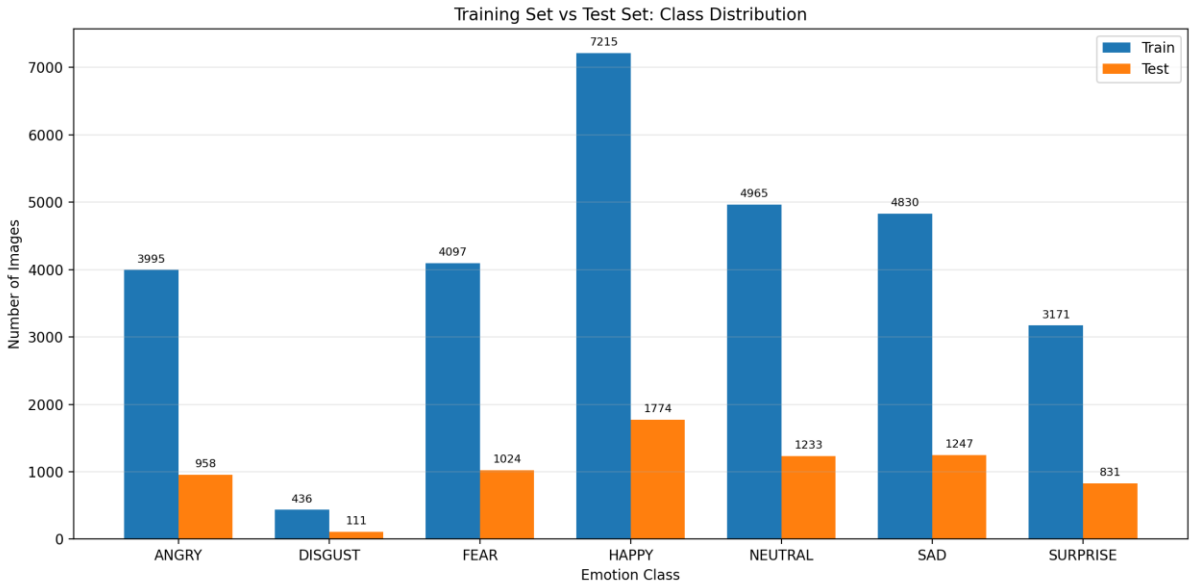

Figure 1. Class distribution of the FER2013 training and test sets.

3.3 Image visualisation and normalisation

Figure 2 illustrates the greyscale nature and low resolution of the FER2013 images. The top row shows raw pixel-value displays, while the bottom row shows normalised versions. Normalisation changes the numerical scale rather than the visual structure of the images, so the examples look visually similar even though the pixel values used by the model differ. The figure also shows why subtle expressions may be difficult: facial details such as eyelid tension, small mouth-corner changes, and eyebrow position must be inferred from a very small image grid.

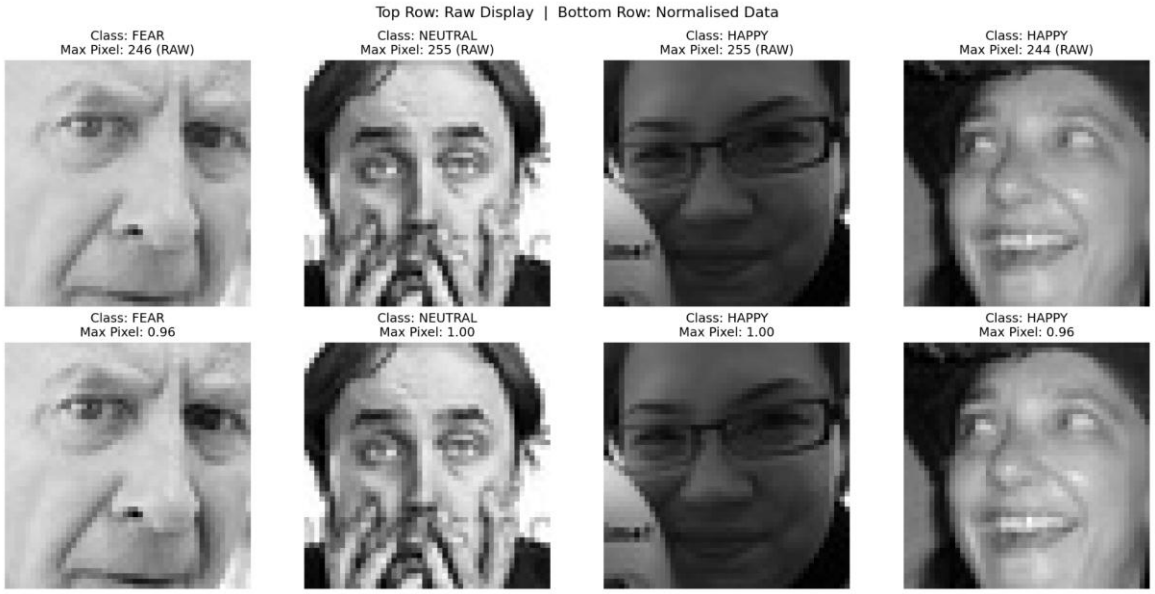

Figure 2. Example FER2013 greyscale images before and after normalisation.

Images were loaded using TensorFlow/Keras ImageDataGenerator with the configuration ImageDataGenerator(rescale=1.0 / 255). This operation normalises pixel values from the original [0, 255] range to the [0, 1] range. TensorFlow documentation describes rescaling as applying a multiplicative factor to sample data, and the use of 1/255 is a standard way to convert image intensities to a compact floating-point range (TensorFlow, 2024).

No data augmentation was used in the final implementation. This is a central methodological detail. The model was trained on the original images after rescaling, without random rotations, shifts, zooms, brightness changes, or flips. This makes the baseline simpler and easier to interpret, but it likely contributed to overfitting because the model repeatedly observed the same training examples rather than a broader set of transformed examples.

4. Methodology and Implementation Workflow

The project follows a standard supervised image-classification workflow. First, the FER2013 dataset is organised into class-labelled train and test directories. Second, the data are explored through class counts and sample visualisation. Third, Keras ImageDataGenerator loads greyscale images in batches and applies pixel-value normalisation. Fourth, the custom CNN is trained using categorical cross-entropy and Adam optimisation. Fifth, callbacks save the best model and stop training when held-out performance stops improving. Finally, the saved model is evaluated using accuracy, loss curves, and confusion matrix analysis.

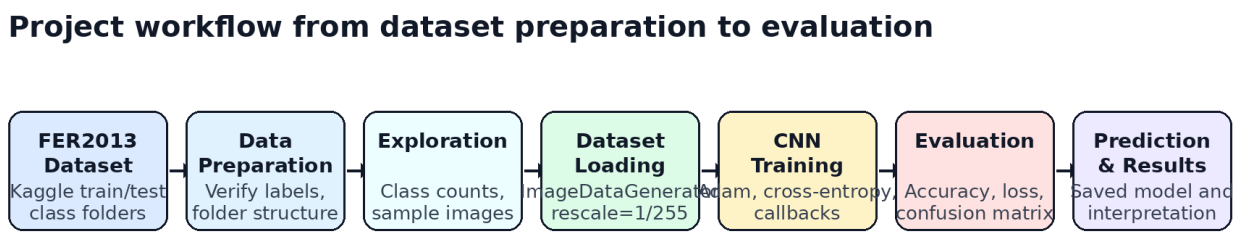

Figure 3. Project workflow from dataset preparation to evaluation.

Figure 3 summarises the pipeline from dataset preparation to final interpretation. The workflow begins with the FER2013 dataset, continues through data preparation and exploration, loads the images into TensorFlow/Keras, trains the CNN, evaluates the saved model, and produces prediction results. This flow is consistent with good practice in supervised learning because it separates data preparation, model development, and evaluation.

The GitHub repository records the implementation materials used for the project, including the training and prediction workflow. The repository is relevant for reproducibility because it allows another user to inspect the code structure and rerun the project with the same dataset organisation. Configuration files such as configs/base.yaml and execution scripts such as run_pipeline.py can support reproducibility by centralising parameters and making the end-to-end workflow easier to repeat. However, the main contribution of the project is the machine learning experiment and its interpretation rather than software engineering complexity.

A key reproducibility issue remains: neural network training is stochastic. Random weight initialisation, mini-batch ordering, and dropout masks can influence the final trajectory. Because no fixed random seed was used, repeated runs may produce slightly different curves and metrics. The final reported performance should therefore be understood as the saved model/evaluation run with 64.41% test accuracy and 1.44 test loss, not as a deterministic value guaranteed in every rerun.

5. CNN Model Architecture

The model is a custom CNN implemented in TensorFlow/Keras. The input shape is 48×48×1, matching the greyscale FER2013 image format. The architecture contains three convolutional blocks followed by a fully connected classification head and a seven-unit Softmax output layer. The filter progression is 64 to 128 to 256, which allows the network to learn increasingly complex features in deeper layers.

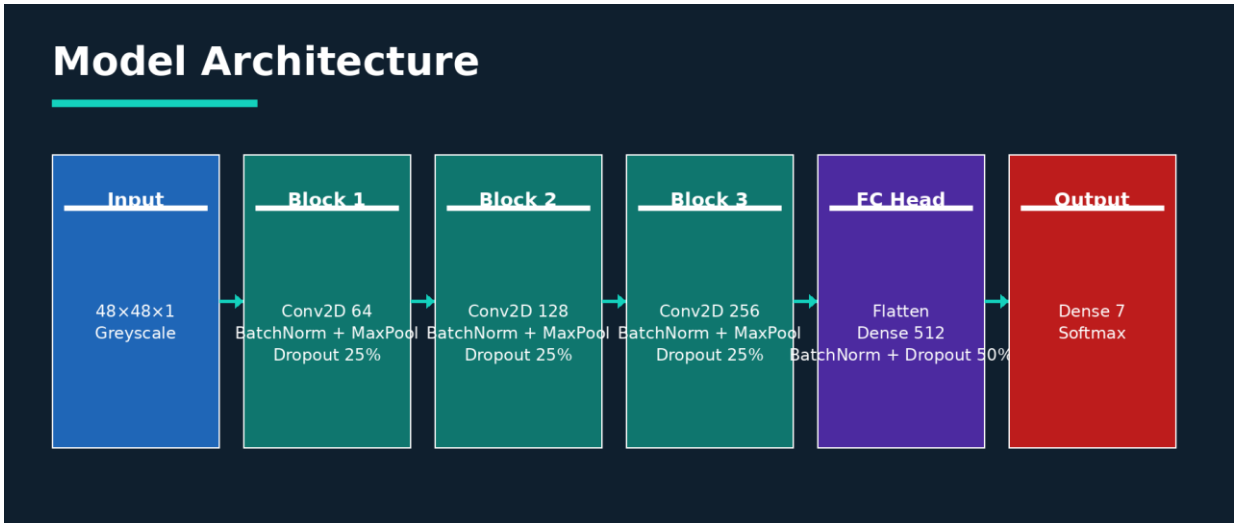

Figure 4. Custom CNN architecture for seven-class facial emotion classification.

Figure 4 presents the architecture at a high level. The input layer receives one-channel greyscale images. Each convolutional block contains Conv2D layers (Two-Dimensional Convolution Layer), Batch Normalisation, MaxPooling, and Dropout. Conv2D layers learn local filters that respond to edges, textures, and facial patterns. Batch Normalisation stabilises intermediate activation distributions and can make training more reliable. MaxPooling reduces spatial size while retaining strong local activations, which helps reduce computation and adds local

translation tolerance. Dropout randomly deactivates a proportion of units during training, reducing co-adaptation and acting as regularisation.

The increasing filter depth reflects the hierarchical nature of CNN representation learning. Early layers need fewer filters because they capture simple features such as edges, contrast changes, and local textures. Deeper layers require more filters because they combine these simple features into more complex facial components and expression-related patterns. This design follows common CNN practice, where spatial resolution decreases while channel depth increases (Goodfellow et al., 2016).

After the convolutional blocks, the model uses Flatten to convert the final feature maps into a vector. A Dense layer then combines the learned spatial features for classification. Batch Normalisation and Dropout are also used in the fully connected head. The final Dense layer has seven units with Softmax activation, producing a probability distribution over the seven FER2013 classes. The predicted class is the class with the highest Softmax probability.

6. Training Configuration

The model was trained as a multi-class classifier using categorical cross-entropy loss. This loss function is appropriate when each example belongs to exactly one class and labels are represented in one-hot form. It penalises the model when it assigns low probability to the correct class, encouraging the predicted Softmax distribution to align with the target label.

The optimiser was Adam. Adam is an adaptive gradient-based optimisation algorithm that estimates first and second moments of gradients to adjust learning rates for individual parameters. Kingma and Ba (2015) introduced Adam as a computationally efficient method that performs well across many deep learning settings. It is widely used in CNN training because it often requires less manual tuning than plain stochastic gradient descent, although learning rate and regularisation still remain important.

Training was configured for a maximum of 50 epochs. EarlyStopping was used with patience = 7, meaning that training would stop if held-out performance failed to improve for seven consecutive epochs. ModelCheckpoint was also used to save the best validation model. This combination is important because the final epoch is not necessarily the best model;

checkpointing preserves the best observed model during training, while early stopping prevents unnecessary continued optimisation after generalisation performance has plateaued.

In the final reported run, training stopped at approximately epoch 47. The model achieved a final saved test accuracy of 64.41% and a test loss of 1.44. Because the model was trained without a fixed random seed, this single run should not be treated as an exact population estimate. A more rigorous experimental design would train the model multiple times and report mean performance with standard deviation.

7. Results and Training Dynamics

Table 2. Final saved evaluation result

Metric	Value	Interpretation
Test accuracy	64.41%	Moderate held-out performance for a custom CNN baseline
Test loss	1.44	Indicates imperfect generalisation and confidence calibration
Approximate training accuracy	Around 90%	Substantially higher than test accuracy
Maximum epochs	50	Upper limit rather than fixed stopping point
Stopping point	Approximately epoch 47	Early stopping ended training after later plateauing

Table 2 reports the final saved evaluation results. The model correctly classified approximately 64.41% of the held-out test images. This is substantially above random guessing for seven classes, but it also means that more than one-third of test images were misclassified. The result is best interpreted as a reasonable custom CNN baseline for a university deep learning project, not as a state-of-the-art system.

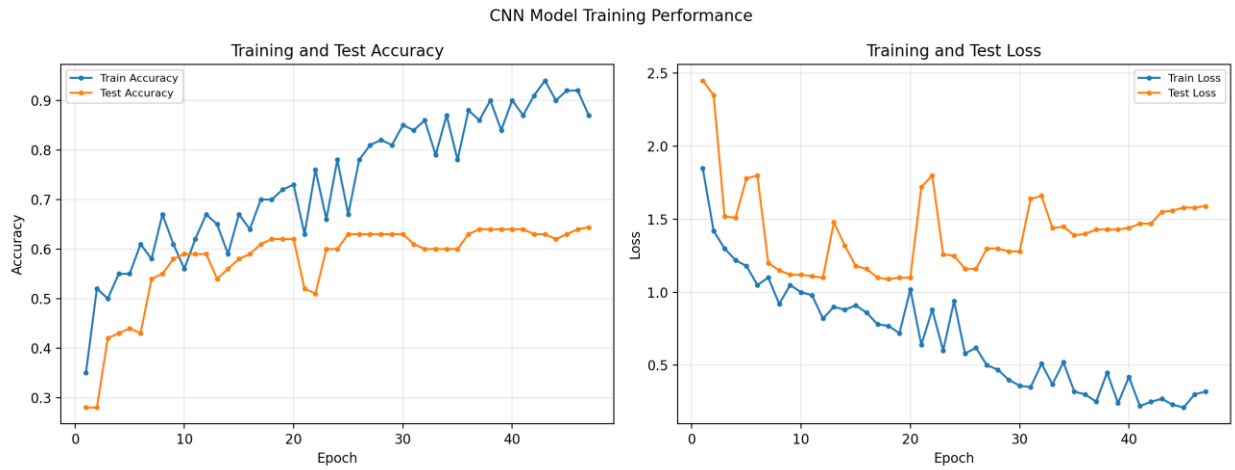

Figure 5. Training and test accuracy/loss curves.

Figure 5 shows the training and held-out test accuracy/loss curves. The curves can be interpreted in four phases. Phase 1, from epochs 1–5, shows rapid learning: training and test accuracy increase sharply while losses fall quickly. This indicates that the model learns broad visual distinctions early in training.

Phase 2, from approximately epochs 5–20, is the healthiest training period. Both training and test accuracy improve together, suggesting that the model is still learning features that generalise beyond the training images. During this phase, the gap between training and test performance is present but not yet dominant.

Phase 3, from approximately epochs 20–35, shows divergence. Training accuracy continues to improve, but test accuracy begins to plateau. This indicates that additional optimisation increasingly benefits the training set more than unseen data. Phase 4, from approximately epochs 35–47, shows clear overfitting. Training accuracy approaches around 90%, while test accuracy remains around 64%, and test loss rises gradually. Early stopping therefore prevents unnecessary continued training after generalisation performance has stopped improving.

The important finding is not that every repeated run will show identical curves. Because no fixed random seed was used, repeated runs may differ due to random initialisation, batch ordering, and dropout. The consistent trend is that training accuracy rises strongly, held-out accuracy plateaus, and overfitting appears in later epochs.

8. Confusion Matrix and Per-Class Analysis

Overall accuracy is useful but incomplete. A confusion matrix provides a more detailed account of which classes are recognised and which are confused. In the confusion matrix, rows represent true labels and columns represent predicted labels. Values on the diagonal are correct predictions, while off-diagonal values are classification errors. The normalised matrix expresses these values as proportions within each true class, making it easier to compare classes of different sizes.

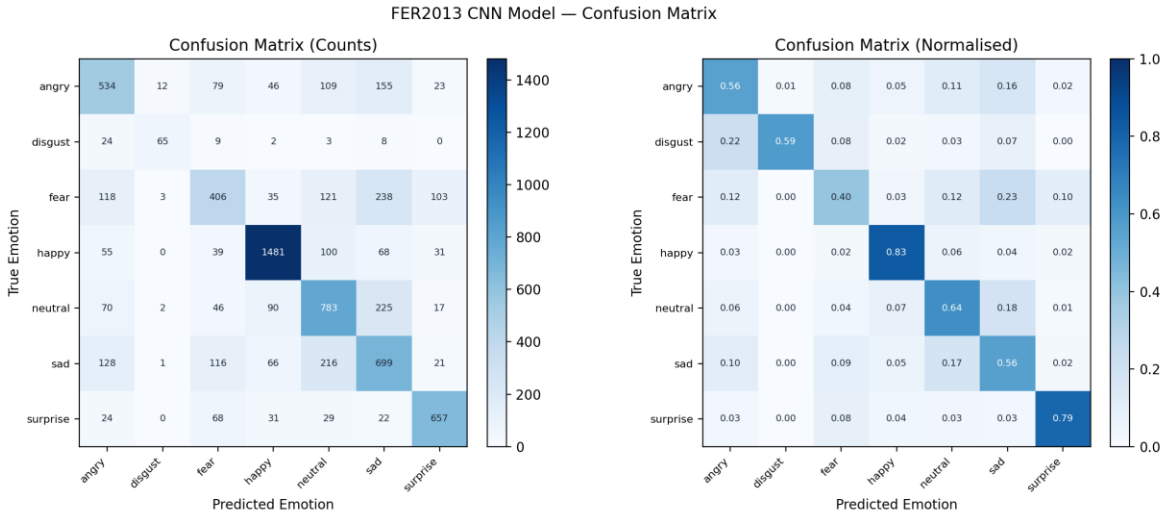

Figure 6. Confusion matrix and normalised confusion matrix for the FER2013 test set.

Figure 6 shows that happy and surprise are the strongest classes. In the normalised confusion matrix, happy is correctly classified at roughly 83–85% depending on rounding, while surprise is correctly classified at roughly 79–81%. These classes are visually distinctive: happy images often contain clear smiles and raised cheeks, while surprise images often include widened eyes, raised eyebrows, and open mouths. Such features produce strong spatial patterns that a CNN can learn from low-resolution greyscale images.

Fear is the weakest class, with recall around 37–40% in the normalised confusion matrix. It is frequently confused with sad, surprise, angry, and neutral. This pattern is plausible because fear shares widened-eye features with surprise, tense facial features with anger, and ambiguous facial configurations with sadness or neutrality. At 48×48 resolution, these differences may be difficult for the model to separate reliably.

Other confusion patterns are also informative. Angry is often confused with sad. Neutral is often confused with sad and happy. Sad is confused with neutral and angry. These errors suggest that the model performs better on expressions with distinctive visual signatures and struggles with subtle, overlapping, or ambiguous expression labels. The confusion matrix therefore supports a cautious interpretation of the final accuracy: the model learned useful patterns, but its performance is uneven across classes, especially overlapping classes.

Table 3. Main class-level findings from the confusion matrix

Class	Observed pattern	Interpretation
Happy	Strongest class, around 83–85% recall	Distinctive smile-related features
Surprise	Strong class, around 79–81% recall	Open-mouth and widened-eye cues
Fear	Weakest class, around 37–40% recall	Confused with sad, surprise, angry, and neutral
Angry	Often confused with sad	Overlapping negative-expression features
Neutral	Often confused with sad and happy	Subtle expression boundaries
Sad	Confused with neutral and angry	Visual ambiguity and label overlap
Disgust	Smallest class	Performance estimates are less stable due to limited samples

9. Discussion

The final test accuracy of 64.41% indicates that the custom CNN learned meaningful visual representations from FER2013. It performs well above chance and shows particularly strong performance on happy and surprise. However, the result must be interpreted carefully. It is a moderate benchmark result for a course-level CNN trained from scratch, not evidence of state-of-the-art performance. Deeper pretrained systems such as VGG-style networks and ResNet architectures have stronger representational capacity and are commonly used as transfer-learning baselines in image-recognition tasks (He et al., 2016; Simonyan & Zisserman, 2015).

The central technical issue is overfitting. Training accuracy reached around 90%, while test accuracy stabilised around 64%. This gap indicates that the model learned patterns specific to the training set that did not fully generalise. The absence of data augmentation likely contributed because the model repeatedly saw the same training examples. The lack of class weighting also meant that the optimiser did not explicitly compensate for class imbalance. Although dropout, batch normalisation, early stopping, and checkpointing helped, they were not sufficient to close the generalisation gap.

The results also illustrate the limits of single-image expression classification. The model cannot use voice, body posture, motion, situational context, or self-report. It only uses 48×48 greyscale facial pixels. Therefore, errors on fear, sad, neutral, and angry are not merely technical failures; they reflect the intrinsic ambiguity of mapping low-resolution facial images to discrete labels.

Reproducibility is another important point. Since no fixed random seed was used, the exact final metric may vary between runs. Different random initialisations and dropout masks can lead to different local optima. The reported result should therefore be stated precisely as the saved

model/evaluation run. A stronger report would include repeated experiments across multiple seeds and report mean accuracy, standard deviation, and per-class variability.

10. Limitations

The project has several limitations. First, no data augmentation was used. This likely increased overfitting because the model did not see transformed variants of the training images. Second, the CNN was trained from scratch. This is valuable pedagogically, but pretrained models such as VGG16, ResNet, or EfficientNet would likely provide stronger visual features. Third, no class weighting or oversampling was applied, leaving the class imbalance uncorrected.

Fourth, no fixed random seed was used, and the final report is based on a single saved run rather than multiple-run averages. This limits statistical confidence in the exact metric. Fifth, FER2013 images are low resolution, making subtle expression differences difficult to capture. Sixth, FER2013 labels are subjective; the model learns annotation patterns rather than objectively measured internal emotions. Seventh, possible demographic and cultural biases are not quantified because the dataset does not provide sufficiently detailed subgroup annotations for fairness analysis.

These limitations do not invalidate the project. Rather, they define the appropriate scope of interpretation. The project provides a complete and academically useful baseline, but it should not be used as evidence that the model is reliable in real-world affective inference settings.

11. Ethical Considerations

Facial expression classification systems raise ethical concerns because they involve human faces and affective labels. The first concern is conceptual validity. The model classifies FER2013 expression labels; it does not directly measure true internal emotions. Treating its output as psychological truth would be scientifically inappropriate and ethically risky, especially in light of debates about the relationship between facial movements and emotional experience (Barrett, 2017; Ekman & Friesen, 1978).

The second concern is privacy and consent. A real-time webcam version of the system could classify people without meaningful awareness or consent. Use in workplaces, classrooms, public spaces, or security settings would require strict transparency, consent, data minimisation, and governance. The third concern is fairness. If FER2013 underrepresents certain demographic or

cultural groups, the model may perform unevenly across populations. Average accuracy can hide subgroup errors.

For these reasons, the model should not be used for high-stakes decisions such as hiring, policing, clinical diagnosis, discipline, or access control. Its appropriate use is educational and experimental. Any practical application would require stronger validation, fairness testing, privacy safeguards, and explicit communication that predictions are uncertain expression-label classifications, not definitive emotional assessments.

12. Future Work

Future work should first add data augmentation. Small rotations, shifts, zooms, and horizontal flips could expose the model to greater visual diversity and reduce overfitting. Augmentation should be applied carefully so that expression-relevant features are not distorted. Second, class weighting or oversampling should be tested to address class imbalance, especially for disgust. Third, fixed random seeds should be used, and multiple experimental runs should be reported to quantify variability.

Fourth, transfer learning should be explored. VGG16, ResNet, and EfficientNet could provide richer pretrained visual features than a CNN trained from scratch. ResNet's residual connections enable very deep architectures to train more effectively (He et al., 2016), while VGG demonstrated the strength of deeper convolutional networks with small filters (Simonyan & Zisserman, 2015). Fifth, explainability methods such as Grad-CAM could help determine whether the model focuses on relevant facial regions such as the eyes, mouth, and eyebrows. Sixth, face alignment could reduce pose variation. Finally, the real-time webcam prediction component could be improved as a demonstration tool, provided that privacy and consent safeguards are explicit.

Future work should be understood in relation to the project's development constraints. Following the non-approval of the team's initial datasets, the final FER2013 implementation was completed within a condensed period of approximately five days. This limited the extent to which alternative architectures and training strategies could be explored. With more time, the project could include a systematic comparison between the current custom CNN and transfer-learning models such as VGG16, ResNet, and EfficientNet. It could also examine data augmentation,

class weighting, oversampling, learning-rate scheduling, explainability methods, and repeated seeded experiments to provide stronger evidence of model generalisation, stability, and class-specific performance.

13. Conclusion

This project implemented a complete CNN-based facial expression classification pipeline using FER2013. The system loads 48×48 greyscale images, normalises pixel values, trains a custom three-block CNN, saves the best model using checkpointing, and evaluates performance using accuracy, loss curves, and confusion matrices. The final saved evaluation run achieved 64.41% test accuracy and 1.44 test loss.

The model learned useful visual patterns, especially for happy and surprise, but showed clear overfitting and class-specific weaknesses. Fear was the weakest class and was often confused with sad, surprise, angry, and neutral. These results are consistent with the visual ambiguity of low-resolution facial expression classification. The model should therefore be described as a FER2013 expression-label classifier, not as a general detector of human emotions.

Overall, the project provides a coherent academic baseline for a Machine Learning / Deep Learning course. It demonstrates the full supervised learning workflow while also revealing important limitations related to overfitting, imbalance, stochastic training, label subjectivity, and ethical interpretation. The clearest path forward is to improve generalisation through augmentation, class balancing, repeated seeded experiments, transfer learning, explainability, and more careful fairness evaluation.

References

Barrett, L. F. (2017). How emotions are made: The secret life of the brain. Houghton Mifflin Harcourt.

Ekman, P., & Friesen, W. V. (1978). Facial action coding system: A technique for the measurement of facial movement. Consulting Psychologists Press.

Goodfellow, I. J., Erhan, D., Carrier, P. L., Courville, A., Mirza, M., Hamner, B., Cukierski, W., Tang, Y., Thaler, D., Lee, D.-H., Zhou, Y., Ramaiah, C., Feng, F., Li, R., Wang, X., Athanasakis, D., Shawe-Taylor, J., Milakov, M., Park, J., et al. (2015). Challenges in

representation learning: A report on three machine learning contests. Neural Networks, 64, 59–63. https://doi.org/10.1016/j.neunet.2014.09.005Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 770–778. https://doi.org/10.1109/CVPR.2016.90

Kaggle. (n.d.). FER-2013. Kaggle. Retrieved May 5, 2026, from https://www.kaggle.com/datasets/msambare/fer2013

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimisation. International Conference on Learning Representations. https://arxiv.org/abs/1412.6980

Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. International Conference on Learning Representations. https://arxiv.org/abs/1409.1556

TensorFlow. (2024). tf.keras.preprocessing.image.ImageDataGenerator. TensorFlow API documentation. https://www.tensorflow.org/api_docs/python/tf/keras/preprocessing/image/ImageDataGenerator